Lesson 1 Overview of C...
PDF 244.63KB

teach me this

Lesson 1: Overview of Computer Graphics

Let me teach you the key concepts from this lesson. I'll break it down into digestible sections.

1. Core Definitions

Term	Definition
Graphics	An image or visual representation of an object
Computer Graphics	The discipline of generating images with the aid of computers
OpenGL	A graphics software system and worldwide standard for 3D computer graphics programming
Pixel	A single point/location in a 2D image array on a screen

2. History (1950s-1980s)

```
text

1950s → Bell Labs, Boeing (scientists & engineers)
1960s-70s → Universities (Ohio State, MIT, Utah, Cornell, UNC, NYIT)
1970s → Xerox PARC
Late 70s-80s → Broadcast video → Major motion pictures
Today → Entertainment companies (Industrial Light & Magic)
```

3. Mathematics & Computer Graphics

Key idea: The world is *analog* (continuous), but computers are *digital* (discrete 0s and 1s). Computer graphics bridges this gap.

Essential math concepts:

- **Linear algebra** (matrices, vectors, linear systems)
- **Cartesian coordinates**: Points as vectors
  - 2D: `p = [x, y]^T`
  - 3D: `q = [x, y, z]^T`

Common operations: Rotate, skew, or scale images using matrix transformations (called *digital image warping*)

4. Raster Displays

The screen is a **2D array of pixels** stored in memory called the **frame buffer**.

The Process:

1. Geometric entities (lines, shapes) are created
2. **Rasterization / Scan conversion** = converting objects into pixel colors and locations
3. This is an *approximation* since the frame buffer is a discrete grid

Display Methods:

Non-interlaced	Interlaced
Pixels displayed row by row (scan line by scan line) at refresh rate	Odd rows and even rows refreshed alternately
Used in computer monitors	Used in commercial TV

Non-interlaced	Interlaced
	At 60Hz, full screen redrawn only 30x/sec (but looks like 60Hz)

5. Basic Line Drawing

Goal: Set pixel colors to approximate a straight line from (x_0, y_0) to (x_1, y_1)

Requirements:

- Straight, passing through endpoints
- Independent of point order (direction doesn't matter)
- Uniformly bright, regardless of slope

The Math

Line equation: $y = mx + b$

Given two points:

```
text

m = (y₁ - y₀) / (x₁ - x₀)
b = y₀ - m·x₀
y = m(x - x₀) + y₀
```

Simple Algorithm (pseudocode)

```
text

m = (y₁ - y₀) / (x₁ - x₀)

for x = x₀ to x₁:
    y = m*(x - x₀) + y₀
    setpixel(x, round(y), linecolor)
```

Note: This algorithm assumes $x_0 < x_1$ and a shallow slope. Real implementations handle all cases (steep slopes, reversed endpoints).

Discussion Questions (Your Assignment)

1. Frame Buffer (half-page response)

The **frame buffer** is a dedicated area of memory that stores the color values for every pixel on the screen. Think of it as a grid matching your display resolution (e.g., 1920×1080). When the graphics system wants to display an image, it reads the frame buffer row by row and sends those values to the display hardware. This happens 60+ times per second (refresh rate). Key points to discuss:

- Relationship between frame buffer size (bits per pixel) and color depth
- How double buffering prevents flicker
- The role of the display controller in scanning the buffer

2. Cathode Ray Tube (CRT)

A CRT works by firing electrons from an electron gun at a phosphor-coated screen. When electrons hit the phosphor, it glows. Electromagnets steer the beam horizontally and vertically to trace the screen line by line. Include a diagram showing:

- Electron gun
- Deflection coils (vertical/horizontal)
- Phosphor-coated screen
- Electron beam path

Quick Check: Do you want me to explain any of these topics in more detail, like the line drawing algorithm with different slope cases, or how frame buffer memory is calculated?

Lesson 2 Mathematical ...

PDF 393.07KB

Lesson 2: Mathematical Background for Computer Graphics

This lesson covers the essential math you need for computer graphics. Let me break it down clearly.

1. Points vs. Vectors

Concept	Definition	Example
Point	A *location* in space	`p = [x, y]^T` (2D) or `q = [x, y, z]^T` (3D)
Vector	A quantity with *direction and magnitude*	`v = [dx, dy]^T`

> **Note:** The superscript `T` means **transpose** (turning a row into a column). Points/vectors are typically written as **column vectors**.

2. Basic Vector Operations

2.3.1 Vector Addition

Add corresponding elements:

```
text

a = [u, v]ᵀ, b = [s, t]ᵀ
a + b = [u+s, v+t]ᵀ
```

2.3.2 Vector Subtraction

```
text
```

```
a - b = [u-s, v-t]ᵀ
```

2.3.3 Vector Scaling

Multiply each element by scalar λ:

```
text

λ·a = [λ·u, λ·v]ᵀ
```

2.3.4 Vector Magnitude (Length)

Using Pythagorean theorem:

```
text

|a| = √(u² + v²)
```

For n-dimensional vector:

```
text

|q| = √(q₁² + q₂² + ... + qₙ²) = √(∑ qᵢ²)
```

2.3.5 Vector Normalization

Make a vector have **length = 1** (unit vector):

```
text

â = a / |a|
```

> The "hat" notation (â) means normalized.

2.3.6 Vector Multiplication (Two Types)

Dot Product (·) → Returns a SCALAR

```
text

a · b = a₁b₁ + a₂b₂ + ... + aₙbₙ = ∑ aᵢbᵢ
```

Three key uses:

1. Compute squared magnitude: $|a|^2 = a \cdot a$
2. Measure angle between vectors: $a \cdot b = |a||b| \cos \theta$
3. Check if vectors are perpendicular (dot = 0)

Cross Product (×) → Returns a VECTOR (3D only)

For $a = [ax, ay, az]^T$ and $b = [bx, by, bz]^T$:

```
text

a × b = [ ay·bz - az·by ,
          az·bx - ax·bz ,
          ax·by - ay·bx ]ᵀ
```

Properties:

- Result is **perpendicular** to both a and b
- Magnitude = area of parallelogram = $|a||b| \sin \theta$
- Used to compute **normals** (vectors perpendicular to surfaces)

3. Matrix Algebra

A matrix is a rectangular array of numbers (rows × columns).

Operations

Operation	How it works
Addition	Add corresponding elements (same size only)

Operation	How it works
Scaling	Multiply every element by scalar
Multiplication	Row × column dot product (complex, see below)
Transpose (A^T)	Swap rows and columns
Identity (I)	Like "1" for matrices: IA = A
Inverse (A^{-1})	$AA^{-1} = A^{-1}A = I$

Matrix Multiplication Rule

For `A (m×n)` and `B (n×p)` → result is `(m×p)`

Element at (i,j) = **row i of A** dot **column j of B**

4. Practice Problems (Your Assignment)

Problem 1: Cross Product

Calculate `a × b` where `a = (2, 3, 4)` and `b = (5, 6, 7)`

Solution:

```
text

a × b = [ (3×7 - 4×6),
          (4×5 - 2×7),
          (2×6 - 3×5) ]
      = [ (21 - 24),
          (20 - 14),
          (12 - 15) ]
      = [ -3, 6, -3 ]
```

Answer: `a × b = (-3, 6, -3)`

Problem 2: Matrix Inverse & Transpose

You need to find the **inverse** and **transpose** of the matrices shown in the lesson.

Quick reminders:

Transpose (A^T): Swap rows ↔ columns

```
text

If A = [a b; c d], then Aᵀ = [a c; b d]
```

Inverse (A^{-1}) for 2×2 matrix:

```
text

If A = [a b; c d], then:
A⁻¹ = (1/(ad - bc)) · [d  -b; -c  a]
```

> `(ad - bc)` is called the **determinant**. If determinant = 0, inverse doesn't exist.

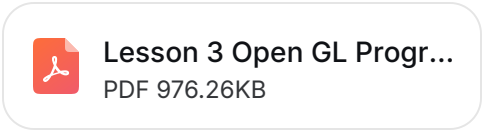
Summary Diagram

```
text

VECTORS                          MATRICES
─────────                        ─────────
+ Addition                       + Addition
- Subtraction                    + Scaling
× Scaling                        × Multiplication (row·col)
|v| Magnitude (length)           ᵀ Transpose (swap rows/cols)
â Normalize (length = 1)         I Identity (no effect)
                                 A⁻¹ Inverse (undoes A)

DOT PRODUCT (·) → scalar
CROSS PRODUCT (×) → perpendicular vector
```

Quick check: Want me to work through a full example of matrix inversion with actual numbers, or explain why the cross product is used for finding surface normals in 3D graphics?



Lesson 3: OpenGL Programming

This lesson introduces OpenGL, the industry-standard API for 3D graphics programming. Let me break it down.

1. What is OpenGL?

OpenGL (Open Graphics Library) is a worldwide standard for computer graphics programming.

Key Characteristics:

Feature	Description
API	Application Programming Interface (specifies how software components interact)
Platform-independent	Works across different computer hardware
Portable	Same code can run on different systems
Implementation needed	You need an OpenGL implementation on your system to create graphics

History & Versions:

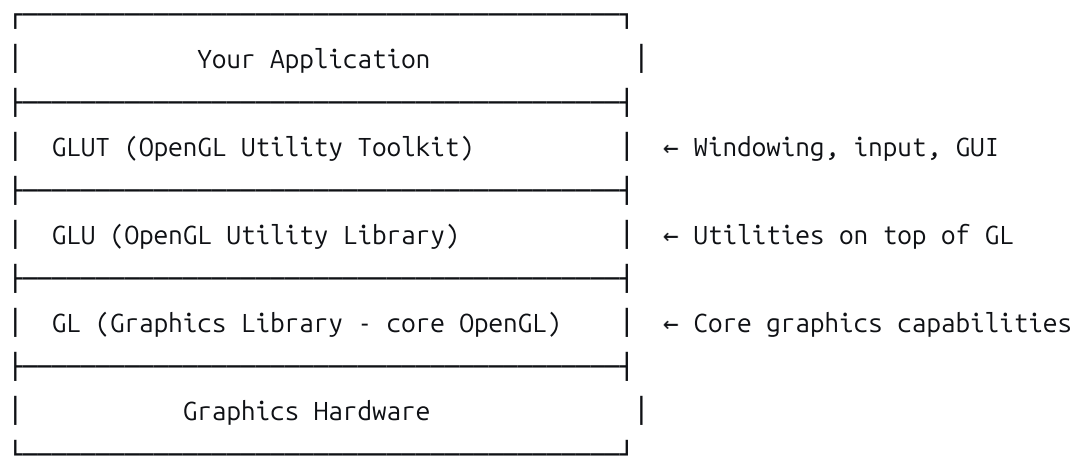
Version	Year	Key Changes
OpenGL 1.0	1992	First version, fixed functionality pipeline
OpenGL 1.1	1997	Pixel blending, texture effects
OpenGL 2.0	2004	**Programmable pipeline** + GLSL (shading language)
OpenGL 3.0	2008	Fixed function pipeline deprecated
OpenGL 4.0	2010	Access to modern GPU features

Specialized Versions:

- **OpenGL ES** → For embedded systems (mobile phones, Raspberry Pi, Android, iOS)
- **WebGL** → OpenGL ES 2.0 for web browsers (JavaScript + HTML5 canvas)

2. OpenGL Library Organization

text



> **Note:** All GLUT function names start with `glut` (e.g., `glutCreateWindow`, `glutDisplayFunc`)

3. Capabilities & Features of OpenGL

3D Objects:

- Lines, polygons, triangle meshes
- Spheres, cubes, quadric surfaces
- NURBS curves and surfaces

3D Modelling & Viewing:

- Transformation functions
- Virtual camera system

Rendering Quality:

- Hidden-surface removal
- Multiple light sources
- Material types, transparency
- Textures, blending, fog

Other Features:

- Display lists (graphics caches)
- Interactive object picking
- Pixel manipulation (anti-aliasing, motion blur, depth of field, soft shadows)

4. How OpenGL Works (5 Steps)

```
text

1. Specify geometric objects
         ↓
2. Describe object properties (color, light reflection)
         ↓
```

```
3. Define how objects should be viewed (camera position, camera type)
        ↓
4. Specify light sources (where, what kind)
        ↓
5. Move camera or objects for animation
```

Application Areas:

- CAD (Computer-Aided Design)
- Virtual reality
- Scientific visualization
- Flight simulation
- Video games

5. Classical Viewing & Projections

Three Basic Elements of Viewing:

1. **One or more objects** (the scene)
2. **A viewer with a projection surface** (camera + screen)
3. **Projectors** (lines from objects to projection surface)

Types of Projections:

Parallel Projection

- Preserves distances and angles
- Shapes preserved
- Good for measurements, building plans, manuals
- **Disadvantage:** Doesn't look real (far objects same size as near objects)
- Used in CAD applications

Perspective Projection

- Looks realistic (far objects appear smaller)
- Parallel lines appear to converge
- **Disadvantage:** Angles and distances not preserved

Axonometric Projections

- Projection plane can move relative to object
- Can see three principal faces of a box-like object
- Lines are scaled (foreshortened)
- Circles become ellipses if not parallel to projection plane
- Some optical illusions possible

6. Homogeneous Coordinates

This is a **critical concept** for computer graphics transformations.

The Problem:

2D transformations (translation, rotation, scaling) are normally done with 2×2 matrices. But **translation** doesn't fit nicely into 2×2 matrix multiplication.

The Solution:

Add an extra dummy coordinate **W** → Convert 2×2 matrices to 3×3 matrices

```
text

Standard point: P(x, y)
Homogeneous point: P(x, y, W)
```

Rules:

Condition	Interpretation
W ≠ 0	Divide by W to get Cartesian: (x/W, y/W, 1)
W = 0	Point is "at infinity" (direction vector)
Multiple representation	(2,5,3) and (4,10,6) represent the **same** point

Why Homogeneous Coordinates?

- **All transformations** (translation, rotation, scaling) can be represented as matrix multiplications
- Transformations can be **combined** (composed/concatenated) into a single matrix

Matrix Composition Example:

```
text

P' = T(dx₁, dy₁) · P
P'' = T(dx₂, dy₂) · P'
P'' = T(dx₁, dy₁) · T(dx₂, dy₂) · P = T(dx₁+dx₂, dy₁+dy₂) · P
```

7. Practice Problem (Car & Side Mirror)

Problem: You sit in a car. The side mirror is 0.4m to your right and 0.3m in front. You:

1. Drive 5m forward
2. Turn 30° right
3. Drive 5m forward again
4. Turn 45° right and stop

Question: What is the position of the side mirror relative to your starting position?

Approach (using matrix transformations):

1. Represent the mirror's initial position as a **vector** or **homogeneous coordinate**
2. Each movement = **translation matrix**
3. Each turn = **rotation matrix**
4. Multiply all transformation matrices together (compose them)
5. Apply the composite matrix to the initial mirror position

Useful Trigonometric Values:

Angle	Cos	Sin	Tan
30°	√3/2 ≈ 0.866	1/2 = 0.5	1/√3 ≈ 0.577
45°	√2/2 ≈ 0.707	√2/2 ≈ 0.707	1
60°	1/2 = 0.5	√3/2 ≈ 0.866	√3 ≈ 1.732

Summary Diagram

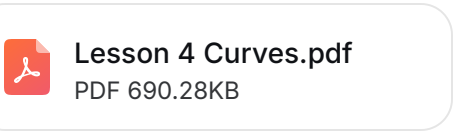
```
text

OpenGL Architecture:

    [Your Code]
         ↓
┌─────────────┐
│  GLUT   │ ← Window management, keyboard, mouse
├─────────────┤
│   GLU   │ ← Utilities (quadrics, NURBS)
├─────────────┤
│   GL    │ ← Core rendering (primitives, transformations)
├─────────────┤
│  GPU    │ ← Hardware
└─────────────┘
```

Quick check: Would you like me to:

1. Show actual code for a basic OpenGL program using GLUT?
2. Explain rotation and translation matrices in more detail?
3. Work through the car mirror problem completely?



Lesson 4: Curves & Polygons

This lesson covers how curves and surfaces are represented in computer graphics, plus polygons, scan conversion, and hidden surface removal.

1. Parametric Curves

Instead of `y = f(x)` (which can only give one y per x), parametric curves use an independent parameter **t**:

```
text

x = f(t)
y = g(t)
z = h(t)

where t ranges from 0 to 1
```

Bézier Curves

Developed by **Pierre Bézier** for Renault automobile bodies. Very widely used in CAD and graphics packages.

Properties of Bézier Curves:

Property	Meaning
Passes through first control point	$p(0) = p_0$

Property	Meaning
Passes through last control point	$p(1) = p_n$
Lies within convex hull	Curve stays inside the polygon formed by control points
Blending functions always positive	Weight contributions are non-negative
Blending functions sum to 1	Ensures affine invariance

2. Ellipses

Definition: Set of points where the sum of distances from two foci is constant.

```
text

d₁ + d₂ = constant (for all points on the ellipse)
```

Properties:

- **Major axis** — longest dimension, passes through both foci
- **Minor axis** — shortest dimension, bisects major axis at right angle

Midpoint Ellipse Algorithm:

- Modified from Bresenham's line algorithm
- **Advantage:** Only addition operations in loops → fast, works on all processors
- Can run in parallel on multiple processors

3. Polygons

A polygon = space bounded by edges between vertices in ordered set.

Important Rules:

Polygon Type	Description
Convex	All interior angles < 180° — OpenGL renders these directly
Concave	Has interior angle > 180° — need GLU to convert to convex polygons
Planar	All vertices lie in a single plane (triangles are always planar)

Surface Normal

A vector **perpendicular** to the polygon's plane.

Why we need normals:

- Give polygon a "front" and "back"
- Describe orientation in 3D space
- Essential for lighting calculations, collisions, culling

Finding a surface normal:

```
text

1. Take two sequential edges E₁ and E₂
2. Invert first edge so both emanate from shared vertex
3. N = E₂ × (-E₁)  [cross product]
4. Normalize N to unit length
```

Polygon Representation Methods:

Method	Description	Pros/Cons
Polygon Soup	Huge list of individual polygons	× Waste storage, lose semantics, brute force

Method	Description	Pros/Cons
Meshes	Linked group of polygons sharing vertices	✅ Save storage, retain semantics, easier manipulation
Triangle Strip	N linked triangles using N+2 vertices	✅ Saves memory (vs 3N vertices)
Triangle Fan	Triangles sharing common vertex	✅ N triangles use N+2 vertices
Quad Strip	N linked quads using 2N+2 vertices	✅ Good for terrain

4. Scan Conversion

Definition: Converting polygon edges (world coordinates) into pixels on screen.

Scan-Converting a Line

- Sample true geometry, approximate to nearest pixels
- **Bresenham (1965)** — still used today, uses only integer arithmetic (fast!)

Scan-Converting a Polygon

- Polygon has been transformed through viewing pipeline
- We know (x, y, z) in screen space
- (x, y) = pixel position, z = distance from eye

Sweep-Line Algorithm

```
text

Steps down a pair of edges scanline by scanline
→ Finds start and end of scanline inside polygon
```

```
→ Efficient because edge gradients computed once
→ But uses floating point (needs rounding)
```

Hidden Surface Removal

The problem: From a given viewpoint, some surfaces are behind others.

Two approaches:

Approach	Method	Difficulty
World space	Work out geometry in 3D before drawing	× Extremely hard
Display space	During scan-conversion, check if nearer object maps to same pixel	✅ Standard approach

Z-Buffer (Depth Buffer)

The most common hidden surface removal method.

How it works:

```
text

Initialize: each pixel = background color
Initialize: each Z-buffer entry = MAXDEPTH

for each pixel P from scan-conversion:
    if z-coordinate of P < ZBUFFER[P]:
        compute color of P
        store color in P
        store z-coordinate in ZBUFFER[P]
    else:
        do nothing (pixel is hidden by closer object)
```

⚠️ **Z-fighting (stitching):** When precision is insufficient for pixels with very similar z-values → incorrect rendering.

5. Rendering Curves in OpenGL

OpenGL uses **evaluators** based on Bernstein basis polynomials.

One-Dimensional Evaluator Steps:

```
text

1. Define evaluator: glMap1*()
2. Enable it: glEnable()
3. Evaluate at points: glEvalCoord1() inside glBegin()/glEnd()
```

glMap1f parameters:

```
c

glMap1f(target, t1, t2, stride, order, &control_points[0][0]);
```

Parameter	Meaning
target	Type of data (e.g., GL_MAP1_VERTEX_3)
t1, t2	Range for parameter t
stride	Entries between control points
order	Number of control points
points	Pointer to control point array

Example Code (bezcurve.c):

```
c

GLfloat ctrlpts[4][3] = {
    {-4.0, -4.0, 0.0},
    {-2.0, 4.0,  0.0},
    { 2.0, -4.0, 0.0},
    { 4.0, 4.0,  0.0}
};
```

```
void init(void) {
    glMap1f(GL_MAP1_VERTEX_3, 0.0, 1.0, 3, 4, &ctrlpts[0][0]);
    glEnable(GL_MAP1_VERTEX_3);
}

void display(void) {
    glBegin(GL_LINE_STRIP);
    for (int i = 0; i <= STEPS; i++)
        glEvalCoord1f((GLfloat)i / (GLfloat)STEPS);
    glEnd();
}
```

Using glMapGrid1 + glEvalMesh1 (more efficient):

c

```
void init(void) {
    glMap1f(GL_MAP1_VERTEX_3, 0.0, 1.0, 3, 4, &ctrlpts[0][0]);
    glEnable(GL_MAP1_VERTEX_3);
    glMapGrid1f(STEPS, 0.0, 1.0);  // define grid
}

void display(void) {
    glEvalMesh1(GL_LINE, 0, STEPS);  // draw all at once
}
```

6. Two-Dimensional Evaluators

For surfaces (not just curves). Uses two parameters: **u** and **v**.

Process:

text

```
1. Define evaluator: glMap2*()
2. Enable: glEnable()
3. Invoke: glEvalCoord2() or glMapGrid2() + glEvalMesh2()
```

glMap2f parameters:

```c
glMap2f(target, u1, u2, ustride, uorder, v1, v2, vstride, vorder, &points);
```

- `uorder × vorder` = total control points
- For vertex evaluators, normals are computed analytically if `GL_AUTO_NORMAL` is enabled

Example:

```c
GLfloat ctrlpoints[4][4][3] = {...};  // 4x4 grid of control points
glMap2f(GL_MAP2_VERTEX_3, 0, 1, 3, 4, 0, 1, 12, 4, &ctrlpoints[0][0][0]);
glEnable(GL_MAP2_VERTEX_3);
glEnable(GL_AUTO_NORMAL);
```

Summary Diagram

```text
Curve/Surface Representation Hierarchy:

                    [Parametric Equations]
                           |
                    [Bézier Curves/Surfaces]
                           |
                 ┌─────────┴──────────┐
                 |                    |
          [1D Evaluator]          [2D Evaluator]
          (curves, t only)        (surfaces, u & v)
                 |                    |
          glMap1*()                 glMap2*()
          glEvalCoord1()            glEvalCoord2()
          glEvalMesh1()             glEvalMesh2()
```

Quick check: Want me to:

1. Explain the math behind Bézier blending functions?
2. Show a complete WebGL program for 2D curves as requested in the discussion session?
3. Walk through a polygon scan-conversion example step by step?



Lesson 5 Transformatio...
PDF 512.31KB

Lesson 5: Transformations

This lesson covers how we move, resize, and rotate objects in computer graphics—essential for animation, modeling, and scene composition.

1. What is a Transformation?

Definition: Modifying and repositioning existing graphics by applying rules.

Why Transformations Matter:

Application	Purpose
Change coordinate frames	World, window, viewport, device coordinate systems
Compose objects	Build complex models from simple parts (e.g., articulated robot arm)
Deformation	Create new shapes from existing ones
Animation	Move objects over time

2. Three Classes of Transformations

Class	Properties Preserved	Examples
Rigid Body	Distances & angles	Translation, Rotation
Conformal	Angles only	Translation, Rotation, **Uniform** Scaling
Affine	Parallelism (lines stay lines)	Translation, Rotation, Scaling, Shear, Reflection

> **Affine transformation** = linear mapping that preserves points, straight lines, and planes. Parallel lines remain parallel.

Real-world use of affine transformations:

- Satellite imagery (correct lens distortion)
- Panorama stitching
- Image registration

3. Basic 2D Transformations

3.1 Translation (Moving)

Move an object by `(tx, ty)`:

```
text

x' = x + tx
y' = y + ty
```

OpenGL: `glTranslatef(tx, ty, tz);`

3.2 Rotation (Turning)

Rotate by angle θ around the **origin**:

```
text

x' = x·cosθ - y·sinθ
y' = x·sinθ + y·cosθ
```

To rotate around an arbitrary point (cx, cy):

```
text

x' = cx + (x - cx)·cosθ - (y - cy)·sinθ
y' = cy + (x - cx)·sinθ + (y - cy)·cosθ
```

OpenGL: `glRotatef(angle, x, y, z);` (angle in degrees)

3.3 Scaling (Resizing)

```
text

x' = x · sx
y' = y · sy
```

- `sx, sy > 1` → enlarge
- `0 < sx, sy < 1` → shrink
- `sx = sy` → uniform scaling (preserves shape)
- `sx ≠ sy` → non-uniform scaling (distorts shape)

OpenGL: `glScalef(sx, sy, sz);`

4. Homogeneous Coordinates (Critical Concept!)

The Problem:

- Translation is **addition**: `P' = P + T`
- Rotation & Scaling are **multiplication**: `P' = M · P`

- We want **all transformations to be multiplications** so we can combine them easily.

The Solution:

Add an extra coordinate **W** to make points 3D (for 2D graphics):

```
text

Standard point: (x, y)
Homogeneous point: (x, y, W)
```

Rules:

Condition	Meaning
W ≠ 0	Divide by W to get Cartesian: (x/W, y/W, 1)
W = 0	Point at infinity (direction vector)
(2,4,2) and (1,2,1)	Represent same Cartesian point (1,2)

In homogeneous coordinates, points become:

```
text

Standard: p̄ = [x, y]ᵀ
Homogeneous: p̂ = [x, y, 1]ᵀ
```

Translation as Matrix Multiplication:

```
text

[ x' ]   [ 1  0  tx ] [ x ]
[ y' ] = [ 0  1  ty ] [ y ]
[ 1  ]   [ 0  0  1  ] [ 1 ]
```

Now ALL transformations are matrices! Composition = matrix multiplication.

5. Transformation Matrices (2D Homogeneous)

Transformation	Matrix
Translation	`[1 0 tx; 0 1 ty; 0 0 1]`
Rotation	`[cosθ -sinθ 0; sinθ cosθ 0; 0 0 1]`
Scaling	`[sx 0 0; 0 sy 0; 0 0 1]`
Shear (X)	`[1 shx 0; 0 1 0; 0 0 1]`
Shear (Y)	`[1 0 0; shy 1 0; 0 0 1]`

6. Combining Transformations (Composition)

Important: Order Matters!

Matrix multiplication is **not commutative**: `A·B ≠ B·A`

Example: Translate then Rotate ≠ Rotate then Translate

Standard Order for Composite Transformations:

```
text

1. Translate (move to origin if rotating around a point)
2. Rotate
3. Scale
4. Translate back (if needed)
```

Formula for rotation around arbitrary point (cx, cy):

```
text
```

```
Composite = T(cx, cy) · R(θ) · T(-cx, -cy)
```

Why this order? (right-to-left application):

1. `T(-cx, -cy)` — Move object so pivot point is at origin
2. `R(θ)` — Rotate around origin
3. `T(cx, cy)` — Move back to original position

7. Transformations in OpenGL

Basic Functions:

c

```c
glTranslatef(tx, ty, tz);    // Move
glRotatef(angle, x, y, z);   // Rotate (angle in degrees)
glScalef(sx, sy, sz);        // Scale
```

Important: OpenGL applies transformations in reverse order!

c

```c
// This code:
glTranslatef(5, 0, 0);
glRotatef(45, 0, 0, 1);
glScalef(2, 2, 2);
drawObject();

// Actually applies: Scale → Rotate → Translate
// (Read from bottom to top)
```

Matrix Operations:

c

```
glMatrixMode(GL_MODELVIEW);  // Work with model/view matrix
glLoadIdentity();            // Reset to identity matrix
glPushMatrix();              // Save current matrix
glPopMatrix();               // Restore saved matrix
```

8. Discussion Session Assignment

Task: Develop C code to implement basic transformations in OpenGL.

Sample Code Structure:

c

```c
#include <GL/glut.h>

float angle = 0.0;
float tx = 0.0, ty = 0.0;
float scale = 1.0;

void display(void) {
    glClear(GL_COLOR_BUFFER_BIT);
    glLoadIdentity();                   // Reset matrix

    // Apply transformations (reverse order!)
    glTranslatef(tx, ty, 0.0);          // 3. Move
    glRotatef(angle, 0, 0, 1);          // 2. Rotate
    glScalef(scale, scale, 1.0);        // 1. Scale

    // Draw a triangle
    glBegin(GL_TRIANGLES);
        glVertex2f(-0.5, -0.5);
        glVertex2f( 0.5, -0.5);
        glVertex2f( 0.0,  0.5);
    glEnd();

    glutSwapBuffers();
}

void keyboard(unsigned char key, int x, int y) {
    switch(key) {
```

```
        case 'r': angle += 10; break;    // Rotate
        case 'R': angle -= 10; break;
        case 't': tx += 0.1; break;      // Translate
        case 'T': tx -= 0.1; break;
        case 's': scale += 0.1; break;   // Scale
        case 'S': scale -= 0.1; break;
        case 27: exit(0); break;         // ESC to quit
    }
    glutPostRedisplay();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
    glutInitWindowSize(500, 500);
    glutCreateWindow("Transformations Demo");
    glutDisplayFunc(display);
    glutKeyboardFunc(keyboard);
    glutMainLoop();
    return 0;
}
```

Summary Diagram

```
text

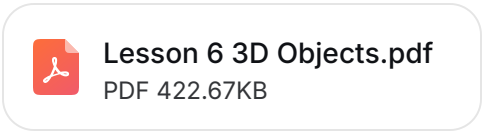
Transformation Hierarchy:

                    [Transformations]
                           |
            ┌──────────────┼──────────────┐
            |              |              |
       [Rigid Body]    [Conformal]     [Affine]
        (distances)     (angles)     (parallelism)
            |              |              |
       Translation     Uniform        Non-uniform
       Rotation        Scaling        Scaling
                                       Shear
                                       Reflection
```

```
All become MATRICES in HOMOGENEOUS COORDINATES
Composition = Matrix Multiplication (order matters!)
```

Quick check: Want me to:

1. Work through a complete example (e.g., rotating a square around its center)?
2. Explain the difference between model, view, and projection matrices?
3. Show how to do hierarchical transformations for articulated objects (like a robot arm)?



Lesson 6: 3D Objects

This lesson covers how we represent, transform, and work with 3D objects in computer graphics.

1. Surface Representations (3 Ways)

Just like with 2D curves, 3D surfaces can be represented in three ways:

Representation	Description	Example
Explicit	`z = f(x, y)` — "height field"	`z = x² + y²` (paraboloid)
Parametric	`p(s, t) = (x(s,t), y(s,t), z(s,t))`	Sphere, torus, Bezier surfaces
Implicit	`f(x, y, z) = 0`	`x² + y² + z² - R² = 0` (sphere)

Parametric Form for Curves (Review):

```
text
```

```
p(s) = x₀ + s·x₁           (linear/line)
p(s) = x₀ + s·x₁ + s²·x₂   (quadratic)
p(s) = Σ sⁱ·xᵢ              (general polynomial)
```

Circle example:

```
text

p(θ) = (r·cosθ, r·sinθ)
```

Parametric Surfaces (2 parameters):

```
text

z = f(x, y)  →  r(x, y) = (x, y, f(x, y))

Surface of revolution (cylinder):
r(x, φ) = (x, R·cosφ, R·sinφ)
```

2. Planes

A plane can be defined by:

- **Three points** (not collinear)
- **Point + normal vector**

Implicit plane equation:

```
text

ax + by + cz + d = 0
```

Where `(a, b, c)` is the normal vector.

3. Surface Tangents and Normals

Concept	Definition	Use
Tangent	Direction that lies along the surface	Curve direction, texturing
Normal	Vector perpendicular to surface	Lighting, shading, culling

For a parametric surface `p(u, v)`:

```
text

Tangent in u direction: ∂p/∂u
Tangent in v direction: ∂p/∂v
Normal = (∂p/∂u) × (∂p/∂v)  (cross product!)
```

4. 3D Affine Transformations

Same principle as 2D, but now in 3D (4×4 homogeneous matrices).

General form: `F(p) = A·p + t` where `A` is 3×3, `p` and `t` are 3D vectors.

Homogeneous 3D Transformation Matrix:

```
text

[ x' ]   [ a b c tx ] [ x ]
[ y' ] = [ d e f ty ] [ y ]
[ z' ]   [ g h i tz ] [ z ]
[ 1  ]   [ 0 0 0 1  ] [ 1 ]
```

Basic 3D Transformations:

Type	Matrix (4×4 homogeneous)
Translation	`[1 0 0 tx; 0 1 0 ty; 0 0 1 tz; 0 0 0 1]`
Scaling	`[sx 0 0 0; 0 sy 0 0; 0 0 sz 0; 0 0 0 1]`

Type	Matrix (4×4 homogeneous)
Rotation about Z	`[cosθ -sinθ 0 0; sinθ cosθ 0 0; 0 0 1 0; 0 0 0 1]`
Rotation about X	`[1 0 0 0; 0 cosθ -sinθ 0; 0 sinθ cosθ 0; 0 0 0 1]`
Rotation about Y	`[cosθ 0 sinθ 0; 0 1 0 0; -sinθ 0 cosθ 0; 0 0 0 1]`

3D Rotation Matrices (detailed):

Rotation about Z-axis (z unchanged, x-y plane rotates):

```
text

q_x = p_x·cosθ - p_y·sinθ
q_y = p_x·sinθ + p_y·cosθ
q_z = p_z
```

Rotation about X-axis (x unchanged, y-z plane rotates):

```
text

q_x = p_x
q_y = p_y·cosθ - p_z·sinθ
q_z = p_y·sinθ + p_z·cosθ
```

Rotation about Y-axis (y unchanged, x-z plane rotates):

```
text

q_x = p_x·cosθ + p_z·sinθ
q_y = p_y
q_z = -p_x·sinθ + p_z·cosθ
```

> **Note:** `det(R) = 1` for pure rotations (no reflection)

5. Spherical Coordinates

A 3D coordinate system using:

Parameter	Description	Range
r	Radial distance from origin	0 to ∞
θ (theta)	Polar angle (from zenith, usually Z-axis)	0 to π
φ (phi)	Azimuthal angle (around Z-axis)	0 to 2π

Conversion to Cartesian:

```
text

x = r·sinθ·cosφ
y = r·sinθ·sinφ
z = r·cosθ
```

Great for: Camera positioning, spherical objects, lighting directions.

6. Nonlinear Transformations

Affine transformations (linear + translation) are first-order. Higher-order deformations create more interesting shapes:

Deformation	Effect
Tapering	Scale varies along an axis (cone shape)
Twisting	Rotation varies along an axis
Bending	Curved deformation around a point/axis

```
text

Example (Tapering along Z):
x' = x · (1 - k·z)
```

```
y' = y · (1 - k·z)
z' = z
```

7. Representing Triangle Meshes

Triangle meshes are the standard way to represent 3D surfaces for rendering.

Efficient Representation:

```
text

Vertex list:    [v0, v1, v2, v3, v4, ...]
Face list:      [(0,1,2), (0,2,3), (1,4,2), ...]
```

Each vertex stored **once** → faces reference vertices by index.

Benefits:

Benefit	Explanation
Memory efficient	Each vertex stored once, not per triangle
Connectivity info	Knows which faces share vertices
Easy to modify	Change one vertex, all faces update
Any polygon	Can decompose to triangles

Storage Comparison:

```
text

Without indexing: N triangles → 3N vertices
With indexing:    N triangles → V vertices + 3N indices
                 (V ≈ N/2 for typical meshes, 75% savings!)
```

Mesh Generation:

Used for:

- Rendering to screen
- Finite element analysis (FEA)
- Computational fluid dynamics (CFD)
- Physical simulation

8. Discussion Session Questions

Question 1: Three purposes of 3D transformations

Purpose	Description	Example
Coordinate transforms	Convert between world, view, and screen spaces	Putting camera anywhere
Shape modeling	Position/rotate/scale parts of complex models	Robot arm joints
Animation	Move objects over time	Car driving, character walking
Camera modeling	Position/orient virtual camera	First-person view

Question 2: Implement a rotation function like glRotate

c

```c
#include <math.h>

typedef struct {
    float x, y, z, w;  // w for homogeneous coordinates
} Vec4;

// Rotation matrix for angle (degrees) around arbitrary axis (rx, ry, rz)
void myRotate(float angle, float rx, float ry, float rz, float matrix[4][4]) {
```

```
    // Initialize to identity
    for(int i = 0; i < 4; i++)
        for(int j = 0; j < 4; j++)
            matrix[i][j] = (i == j) ? 1.0f : 0.0f;

    // Normalize axis
    float len = sqrt(rx*rx + ry*ry + rz*rz);
    if(len == 0) return;
    rx /= len; ry /= len; rz /= len;

    float rad = angle * 3.14159265f / 180.0f;
    float c = cos(rad);
    float s = sin(rad);
    float t = 1 - c;

    // Build rotation matrix (Rodrigues' formula)
    matrix[0][0] = t*rx*rx + c;
    matrix[0][1] = t*rx*ry - s*rz;
    matrix[0][2] = t*rx*rz + s*ry;

    matrix[1][0] = t*ry*rx + s*rz;
    matrix[1][1] = t*ry*ry + c;
    matrix[1][2] = t*ry*rz - s*rx;

    matrix[2][0] = t*rz*rx - s*ry;
    matrix[2][1] = t*rz*ry + s*rx;
    matrix[2][2] = t*rz*rz + c;

    // Bottom row stays [0, 0, 0, 1]
}

// Apply matrix to a vertex
Vec4 applyMatrix(float matrix[4][4], Vec4 v) {
    Vec4 result;
    result.x = matrix[0][0]*v.x + matrix[0][1]*v.y + matrix[0][2]*v.z + matrix[0][3]*v.
w;
    result.y = matrix[1][0]*v.x + matrix[1][1]*v.y + matrix[1][2]*v.z + matrix[1][3]*v.
w;
    result.z = matrix[2][0]*v.x + matrix[2][1]*v.y + matrix[2][2]*v.z + matrix[2][3]*v.
w;
    result.w = matrix[3][0]*v.x + matrix[3][1]*v.y + matrix[3][2]*v.z + matrix[3][3]*v.
w;
```

```
    return result;
}
```

Summary Diagram

```
text

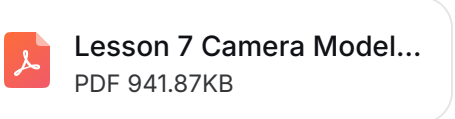
3D Object Representations:

                    [3D Object]
                         |
          ┌──────────────┼──────────────┐
          |              |              |
   [Explicit]       [Parametric]     [Implicit]
   z = f(x,y)       p(u,v)           f(x,y,z)=0
   (height field)   (Bezier, NURBS)  (sphere, plane)
                         |
                    [Triangle Mesh]
                         |
                ┌────────┴────────┐
                |                 |
          [Vertices]          [Faces]
          (positions)         (indices)

Transformations:  Translation → Rotation → Scaling
                   (3D = 4×4 homogeneous matrices)
```

Quick check: Want me to:

1. Explain how to compute surface normals for a triangle mesh?
2. Walk through a complete 3D cube rendering with transformations?
3. Show how to combine multiple transformations in the correct order for 3D?



Lesson 7: Camera Models & Camera Control

This lesson covers how virtual cameras work in computer graphics—from the physics of lenses to the math of projections in OpenGL.

1. Two Camera Models

7.1.1 Thin Lens Model

How real cameras work: A lens focuses light onto a sensory surface (film/sensor).

Key concept—Focal length (|f|): Distance behind the lens where rays from an **infinitely distant** source converge in focus.

Thin Lens Equation:

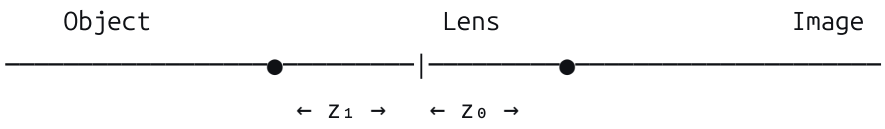
```
text

1/|f| = 1/z₀ + 1/z₁
```

Where:

Variable	Meaning
z_1	Distance from lens center to object
z_0	Distance from lens center to focused image
$\|f\|$	Focal length

```
text
```



7.1.2 Pinhole Camera Model (Idealization)

Simpler model: As aperture shrinks to zero, light travels through a single point (the pinhole).

Characteristics:

- Light travels along **single straight path** through pinhole
- Image is **upside-down** on image plane
- No lens = no focusing issues
- Used by OpenGL (consistent with its camera model)

```
text

    Object                    Pinhole                 Image Plane
         \                    |      /
          \                   |     /
           \                  |    /
            \                 |   /  (upside-down image)
             ●————————————————●—●————————————
                               |
                          Optical axis
                          (negative Z in OpenGL)
```

OpenGL Camera Convention:

- **Right-handed coordinate system**
- X-axis = horizontal → right
- Y-axis = vertical → up
- **Optical axis (gaze direction) = negative Z-axis**

```
text

        Y (up)
        |
        |
        +------ X (right)
       /
      /
     Z (out of screen = behind camera)
```

```
Camera looks down the negative Z axis!
```

2. Camera Projections (Two Main Types)

Type	Description	Characteristics
Parallel Projection	Projectors are parallel	No size change with distance, preserves parallel lines
Perspective Projection	Projectors converge at eye point	Distant objects appear smaller, realistic

Parallel Projection

- Corresponds to camera at **infinite distance**
- **Infinite focal length** ("zoom")
- Lines parallel in 3D remain parallel in 2D
- Used in engineering drawings, CAD

Perspective Projection

- Linear projection onto picture plane
- **Vanishing point(s):** Where parallel lines appear to meet
- Looks realistic (human eye works this way)
- Categories: **1-point, 2-point, 3-point perspective**

```
text

Parallel Projection:          Perspective Projection:
    \    |    /                     \   |   /
     \   |   /                        \ | /
      \  |  /                          \|/
       \ | /                            ●  (eye point)
```

```
\|/
 ● (eye at infinity)
```

3. Projecting a Triangle (5-Step Pipeline)

```
text

OBJECT SPACE → WORLD SPACE → CAMERA SPACE → CLIP SPACE → NDC → SCREEN
```

Step-by-step:

Step 1: Triangle in object coordinates

```
text

p̂₁, p̂₂, p̂₃ (in object's local frame)
```

Step 2: Transform to world coordinates

```
text

p̂_w₁, p̂_w₂, p̂_w₃ = object_transform × p̂_i
```

Step 3: Transform to camera coordinates

```
text

p̂_cᵢ = M_wc × p̂_wᵢ
```

(M_wc = world-to-camera matrix)

Step 4: Homogeneous perspective transformation

```
text

x̂ = M_p × p̂_c
```

Where:

```
text

M_p = [  near/r   0        0                0
         0     near/t      0                0
         0        0   -(far+near)/(far-near)   -2·far·near/(far-near)
         0        0       -1                0 ]
```

Step 5: Divide by last component (Perspective Division)

```
text

x = x̂/x̂_w,  y = ŷ/x̂_w,  z = ẑ/x̂_w
```

Result = Normalized Device Coordinates (NDC) in range [-1, 1]

4. Camera Projections in OpenGL

Setting Up Projection Matrix:

```
c

glMatrixMode(GL_PROJECTION);
glLoadIdentity();
```

Perspective Projection (glFrustum):

```
c

glFrustum(left, right, bottom, top, near, far);
```

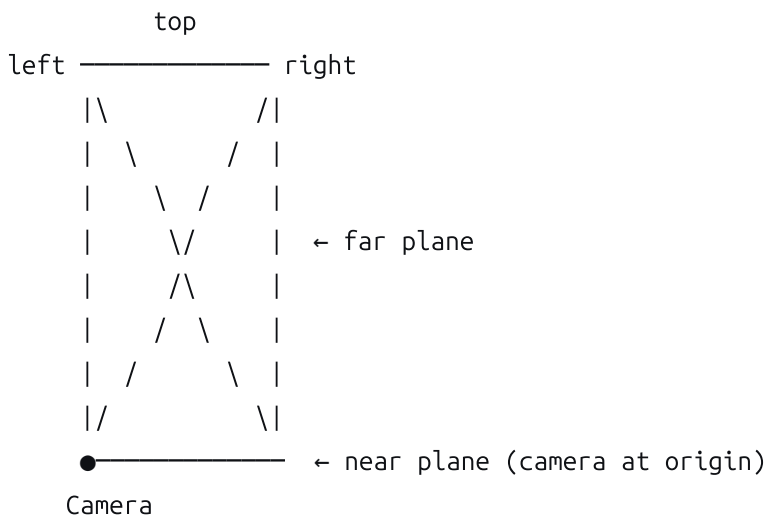
Parameters:

Parameter	Meaning
left, right	X extents of near plane

Parameter	Meaning
bottom, top	Y extents of near plane
near	Distance to near plane (>0)
far	Distance to far plane (> near)

Visualizing the viewing frustum:

```
text
```



Orthographic Projection (glOrtho):

```
c
glOrtho(left, right, bottom, top, near, far);
```

- No perspective effect (parallel projection)
- Objects same size regardless of distance

GLU Helper Function (easier perspective):

```
c
gluPerspective(fovy, aspect, near, far);
```

Parameter	Meaning
fovy	Field of view angle in Y direction (degrees)
aspect	Width/height ratio of viewport
near, far	Near and far clip planes

5. Discussion Session Questions

Question 1: Creating an Environment Map with Home Tools

Method using a reflective sphere (chrome ball):

1. **Materials needed:** Camera, shiny chrome/metal ball, tripod
2. **Steps:**
   - Place chrome ball in scene
   - Take photos from multiple angles (or one shot if ball captures full environment)
   - Extract reflection from ball surface
   - Unwrap spherical image to 2D texture map
3. **Alternative (mirror ball method):**
   - Photograph a mirrored sphere
   - Convert circular reflection to rectangular environment map mathematically

Question 2: Advantages & Disadvantages

Aspect	Thin Lens	Pinhole Camera
Light gathering	✅ Good (large aperture)	× Poor (very small aperture)
Exposure time	✅ Short	× Long (need bright light or long exposure)

Aspect	Thin Lens	Pinhole Camera
Sharpness	× Can have focus issues	✅ Infinite depth of field (everything in focus)
Distortion	× Lens distortion possible	✅ No lens distortion
Complexity	× Complex to model	✅ Simple to model
Realism in CG	✅ More realistic (lens effects)	× Too perfect (no depth of field)

Question 3: Projection Types Explained

Projection	Description	Characteristics
Orthographic	All projectors perpendicular to image plane	No foreshortening, preserves scale
Multiview	Multiple orthographic views (front, top, side)	Standard in engineering drawings
Axonometric	Parallel projection with viewing plane tilted	Shows 3 faces, lines foreshortened
Isometric	Angles between axes = 120°	All 3 axes scaled equally
Dimetric	Two axes scaled equally, third differently	Moderate realism
Trimetric	All three axes scaled differently	Most flexible, least realistic
Oblique	Projectors not perpendicular to image plane	Front face true shape, sides receding

Visual summary:

```
text

 Orthographic:   Isometric:      Perspective:
    □              ◇                □
```

```
  □ □          ◇ ◇          ◇  □
 □ □ □        ◇ ◇ ◇        ◇    □
(no depth)   (3D-like)    (vanishing point)
```

Summary Diagram

```
text

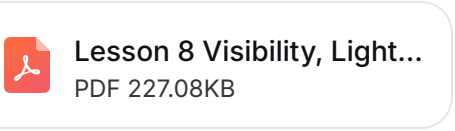
Camera Pipeline:

    [3D Point]
         ↓
    Object Coordinates (local)
         ↓ Model-View Matrix
    World Coordinates
         ↓ View Matrix
    Camera Coordinates (eye space)
         ↓ Projection Matrix
    Clip Coordinates (homogeneous)
         ↓ Perspective Division
    Normalized Device Coordinates [-1,1]
         ↓ Viewport Transform
    Screen Coordinates (pixels)

OpenGL Matrices:
    GL_MODELVIEW  = View × Model
    GL_PROJECTION = Projection matrix
```

Quick check: Want me to:

1. Derive the perspective projection matrix step by step?
2. Show how to implement a first-person camera with keyboard/mouse control?
3. Explain the math behind environment map creation?



Lesson 8: Visibility, Lighting, Reflections & Shading

This lesson covers how we determine what's visible, how light interacts with surfaces, and how to shade 3D objects realistically.

1. Visibility (Hidden Surface Removal)

The problem: When multiple opaque polygons cover the same screen space, only the frontmost one should be visible.

8.2 Visibility in OpenGL

OpenGL primarily uses **depth buffering (Z-buffer)** for visibility.

Setup Steps:

```c
// 1. Initialize with depth buffer
glutInitDisplayMode(GLUT_RGB | GLUT_DOUBLE | GLUT_DEPTH);

// 2. Enable depth testing
glEnable(GL_DEPTH_TEST);

// 3. Clear depth buffer each frame
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
```

Additional Functions:

Function	Purpose
`glDepthMask(GL_FALSE)`	Disable depth writes (testing still occurs)
`glDepthMask(GL_TRUE)`	Re-enable depth writes

Advanced Visibility Techniques:

- **BSP Trees** (Binary Space Partitioning): Used in games to cull invisible geometry

- **Back-to-front drawing**: For translucent objects (without writing to depth buffer)

2. Simple Reflection Models

Dichromatic Reflection Model

```
text

Observed Image Color = α × Body Color + β × Specular Reflection Color
```

Component	Description
Body Color (Diffuse)	Light scattered evenly in all directions (matte surfaces)
Specular Reflection	Light reflected in a specific direction (shiny highlights)

Surface reflection depends on **both viewing direction** and **illumination direction**.

3. Lighting in OpenGL

OpenGL implements the **Phong reflection model**.

Four Steps to Set Up Lighting:

```
text

1. Enable lighting and select shading model
                    ↓
2. Specify normals for each vertex
                    ↓
3. Specify material properties (color, shininess)
                    ↓
4. Specify light sources (position, color, intensity)
```

Enable Lighting:

```c
glEnable(GL_LIGHTING);   // Global enable
// Once enabled, glColor*() is IGNORED!
```

Enable Individual Light Sources:

```c
// Up to 8 lights: GL_LIGHT0 through GL_LIGHT7
glEnable(GL_LIGHT0);
glEnable(GL_LIGHT1);
// etc.
```

Light Source Properties:

```c
// Position (w=1 for point light, w=0 for directional)
GLfloat position[] = {1.0f, 2.0f, 3.0f, 1.0f};
glLightfv(GL_LIGHT0, GL_POSITION, position);

// Colors
GLfloat ambient[]  = {0.2f, 0.2f, 0.2f, 1.0f};  // Low-level background
GLfloat diffuse[]  = {0.8f, 0.8f, 0.8f, 1.0f};  // Main color
GLfloat specular[] = {1.0f, 1.0f, 1.0f, 1.0f};  // Highlights
glLightfv(GL_LIGHT0, GL_AMBIENT, ambient);
glLightfv(GL_LIGHT0, GL_DIFFUSE, diffuse);
glLightfv(GL_LIGHT0, GL_SPECULAR, specular);
```

Material Properties:

```c
GLfloat material_ambient[]  = {0.2f, 0.5f, 0.2f, 1.0f};
GLfloat material_diffuse[]  = {0.5f, 0.8f, 0.5f, 1.0f};
GLfloat material_specular[] = {0.8f, 0.8f, 0.8f, 1.0f};
GLfloat material_shininess[] = {100.0f};  // 0-128, higher = sharper highlight
```

```
glMaterialfv(GL_FRONT, GL_AMBIENT, material_ambient);
glMaterialfv(GL_FRONT, GL_DIFFUSE, material_diffuse);
glMaterialfv(GL_FRONT, GL_SPECULAR, material_specular);
glMaterialfv(GL_FRONT, GL_SHININESS, material_shininess);
```

Limitations of OpenGL Lighting:

- Pipeline is **one-way** → no mirrors, no inter-reflections
- Emissive materials (glow sticks) don't illuminate other objects
- Can only model direct lighting (light source → object → eye)

4. Shading Methods

Comparison Table:

Method	How it works	Speed	Quality
Flat Shading	One color per polygon	Fastest	Lowest (faceted look)
Gouraud Shading	Colors at vertices interpolated across polygon	Medium	Medium
Phong Shading	Normals interpolated, lighting per pixel	Slowest	Highest

8.5.1 Flat Shading

Characteristics:

- Simplest, cheapest, fastest
- One color intensity fills entire polygon
- Good for objects truly made of flat faces

OpenGL:

```c
glShadeModel(GL_FLAT);
// Color at first vertex determines whole polygon color
```

Mach Band Effect: Human eyes accentuate intensity discontinuities at polygon boundaries → faceted appearance.

8.5.2 Gouraud Shading (Interpolative)

How it works:

1. Compute lighting (radiance) at each vertex
2. Linearly interpolate colors across triangle
3. Fill pixels with interpolated colors

OpenGL (default):

```c
glShadeModel(GL_SMOOTH);  // Gouraud shading
```

Pros: Efficient, smooth appearance
Cons: Lower accuracy than Phong, can miss specular highlights inside polygons

8.5.3 Phong Shading

How it works:

1. Interpolate **normals** (not colors) across triangle
2. Compute lighting equation at **each pixel**
3. Most accurate results

OpenGL support: Not directly in fixed-function pipeline. Requires **pixel shaders** on programmable hardware.

Advanced: Can use a **normal map** to assign arbitrary normals within faces.

5. Shading in OpenGL Summary

```c
// Flat shading
glShadeModel(GL_FLAT);

// Gouraud shading (default, smooth)
glShadeModel(GL_SMOOTH);

// Phong shading - requires programmable shaders (GLSL)
// Not available in fixed-function OpenGL
```

Shading Pipeline Visualization:

```text
Flat:        Vertex → [Lighting] → Color → Polygon (uniform)
Gouraud:     Vertex → [Lighting] → Color → Interpolate → Fill
Phong:       Vertex → Interpolate Normal → [Lighting per pixel] → Fill
```

6. Discussion Session Question

Question: How will you create an environment map to capture lighting effects from a real-world outdoor scene? (Think of Paul Debevec's "Rendering with Natural Light" scene.)

Answer: Mirror Ball Method (Light Probe)

Materials needed:

- Camera (any digital camera)
- Shiny chrome/mirrored sphere (ornament or polished metal ball)

- Tripod

Steps:

1. **Photograph the mirror ball**
   - Place ball in outdoor scene (sunny day ideal)
   - Camera on tripod, positioned to see full sphere
   - Take multiple exposures for High Dynamic Range (HDR)
2. **Extract the environment**
   - The sphere reflects entire hemisphere (360° around)
   - Polar coordinate mapping: angle around ball = azimuth, distance from center = elevation
3. **Convert to latitude/longitude map**
   - Unwrap circular reflection to rectangular texture
   - Formula: `u = atan2(y, x) / (2π)`, `v = acos(z) / π`
4. **Use in CG scene**
   - Map extracted image as environment map
   - Apply to reflective objects for realistic lighting

Paul Debevec's contribution: Used this technique to capture "Light Probes" that accurately relight CG objects into real-world scenes (e.g., "Rendering with Natural Light" - the church scene).

Simplified Home Version:

```python
# Pseudo-code for unwrapping
for each pixel (u,v) in output map:
    theta = u * 2π      # azimuth
    phi   = v * π       # elevation (0 at poles)

    # Find corresponding point on sphere reflection
    x = sin(phi) * cos(theta)
    y = sin(phi) * sin(theta)
```

```
        z = cos(phi)

        # Sample from photograph of mirror ball at (x,y)
        output[u][v] = sample_from_ball_image(x, y)
```

Summary Diagram

```
text

Visibility → Lighting → Shading → Output
     |           |          |
     ▼           ▼          ▼
  Z-buffer    Phong      Flat/Gouraud/Phong
  BSP Tree    Model      (ShadeModel)

Lighting Pipeline:
┌──────────┐    ┌──────────┐    ┌──────────┐
| Material |    |  Light   |    | Normal   |
| (color,  | +  | (color,  | ×  | (direction) | = Pixel Color
| shininess)|   | position)|    |          |
└──────────┘    └──────────┘    └──────────┘
```

Quick check: Want me to:

1. Derive the Phong lighting equation mathematically?
2. Show a complete OpenGL program with lighting and shading?
3. Explain how normal mapping works in pixel shaders?



Lesson 9 Texture Mappi...
PDF 212.84KB

Lesson 9: Texture Mapping

This lesson covers how to "wrap" images onto 3D surfaces to add detail without increasing geometric complexity.

1. What is Texture Mapping?

Texture: The appearance and feel of a surface. In computer graphics, a texture is an image used to define surface characteristics.

Texture Mapping: The process of mapping an image onto a 3D surface.

Why Use Texture Mapping?

Problem	Solution
Too much detail to model geometrically (grass, brick, fabric)	Use a texture image instead
Pattern repeats (tiling floor, wallpaper)	Wrap texture repeatedly
Realistic appearance needed	Paint image onto surface

Key Terminology:

Term	Definition
Texel	A single element in a texture array (like pixel in an image)
Texture Unit (TMU/TPU)	GPU hardware component that does sampling
Texture Object	Holds the texture data
Sampling	Computing a color from texture coordinates

2. Texture Sources

Two Ways to Define Texture Data:

Method	Description
Hard-code	Manually specify texel values in array
Load image	Read JPG, PNG, GIF, BMP into texture array

Types of Textures:

Type	Dimensions	Use Case
2D Texture	2D array of colors (u, v)	Most common (paint on surfaces)
3D Texture	Volume of colors (u, v, w)	Solid textures (marble, wood grain)
Environment Map	Spherical/reflection mapping	Skyboxes, reflections

3. Texture Mapping Techniques

Technique	Description
2D Texture Mapping	Paint 2D pattern onto surface
Environment Mapping	Reflect surroundings (chrome ball effect)
Bump Mapping	Perturb normals to simulate bumps (without geometry change)
Procedural Texture	Generated mathematically (not from an image)

Coordinate Systems:

```
text

Texture Space (u, v) → Surface Space → Screen Space (x, y)
     (0,1)                  (0,1)            (pixels)
```



4. Textures and Phong Reflectance

Texture mapping can modify different components of the Phong model:

```
text

Phong Model Output = Ambient + Diffuse + Specular

Texture can replace ANY of these:
- Diffuse color (most common: "painted" look)
- Specular color (glossy patterns)
- Normals (bump mapping)
- Transparency (alpha textures)
```

Phong Model Components Recap:

Component	Inputs
Ambient	`k_a × L_a` (material × light)
Diffuse	`k_d × L_d × (n·l)`
Specular	`k_s × L_s × (n·h)^shininess`

5. Aliasing

What is Aliasing?

The phenomenon that makes smooth lines and edges appear **stair-stepped or jagged** (the "jaggies").

Types:

Type	Description	Example
Spatial Aliasing	Jagged edges in still images	Diagonal lines looking like stairs
Temporal Aliasing	Aliasing in animation over time	Wagon wheel appearing to spin backward ("strobing")

Anti-aliasing solutions:

- **Mipmapping** (pre-filtered texture levels)
- **Bilinear/Trilinear filtering**
- **Multisampling (MSAA)**

6. Texturing in OpenGL (6 Steps)

Step 1: Enable Texturing

```c
glEnable(GL_TEXTURE_2D);
```

Step 2: Create/Generate Texture

```c
GLuint textureID;
glGenTextures(1, &textureID);
glBindTexture(GL_TEXTURE_2D, textureID);
```

Step 3: Set Texture Parameters (Wrapping & Filtering)

```c
```

```
// Wrapping (what happens outside [0,1] range)
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);  // u direction
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);  // v direction

// Filtering (minification & magnification)
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
```

Wrapping Options:

Value	Effect
GL_REPEAT	Tile the texture
GL_CLAMP	Clamp to edge color
GL_CLAMP_TO_EDGE	Repeat edge pixels
GL_MIRRORED_REPEAT	Mirror when tiling

Filtering Options:

Value	Effect
GL_NEAREST	Pick nearest texel (blocky)
GL_LINEAR	Bilinear interpolation (smooth)

Step 4: Upload Texture Data

c

```
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, width, height, 0,
             GL_RGB, GL_UNSIGNED_BYTE, imageData);
```

Step 5: Specify Texture Coordinates for Each Vertex

c

```
glBegin(GL_QUADS);
    glTexCoord2f(0.0f, 0.0f);  glVertex3f(-1.0f, -1.0f, 0.0f);
    glTexCoord2f(1.0f, 0.0f);  glVertex3f( 1.0f, -1.0f, 0.0f);
    glTexCoord2f(1.0f, 1.0f);  glVertex3f( 1.0f,  1.0f, 0.0f);
    glTexCoord2f(0.0f, 1.0f);  glVertex3f(-1.0f,  1.0f, 0.0f);
glEnd();
```

Step 6: Clean Up (when done)

c

```
glDeleteTextures(1, &textureID);
```

Complete Example:

c

```
void initTexture(void) {
    // Generate texture
    glGenTextures(1, &texID);
    glBindTexture(GL_TEXTURE_2D, texID);

    // Parameters
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

    // Load data (assuming image loaded into 'data')
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, imgWidth, imgHeight,
                 0, GL_RGB, GL_UNSIGNED_BYTE, data);

    // Enable
    glEnable(GL_TEXTURE_2D);
}

void display(void) {
    glBindTexture(GL_TEXTURE_2D, texID);

    glBegin(GL_QUADS);
        glTexCoord2f(0,0); glVertex3f(-1,-1,0);
```

```
            glTexCoord2f(1,0); glVertex3f( 1,-1,0);
            glTexCoord2f(1,1); glVertex3f( 1, 1,0);
            glTexCoord2f(0,1); glVertex3f(-1, 1,0);
        glEnd();
    }
```

7. Discussion Session Questions

The assignment asks to discuss:

1. DDA Algorithm (Digital Differential Analyzer)

Purpose: Line drawing algorithm

How it works:

```
text

1. Compute dx = x1 - x0, dy = y1 - y0
2. steps = max(|dx|, |dy|)
3. x_increment = dx / steps
4. y_increment = dy / steps
5. x = x0, y = y0
6. For i = 0 to steps:
       setpixel(round(x), round(y))
       x += x_increment
       y += y_increment
```

Pros: Simple, handles any slope
Cons: Floating point operations (slower than Bresenham)

2. Bresenham's Line Algorithm

Purpose: Line drawing using ONLY integer arithmetic

How it works (octant 1, dx > dy > 0):

```
text
```

```
1. dx = x1 - x0, dy = y1 - y0
2. error = 2*dy - dx
3. For x = x0 to x1:
       setpixel(x, y)
       if error > 0:
           y = y + 1
           error = error - 2*dx
       error = error + 2*dy
```

Pros: Fast (integer only), no division

Cons: More complex logic for all octants

Summary Diagram

```
text

Texture Mapping Pipeline:

    [Texture Image]          [3D Model]
         |                        |
    [Texel Array]             [UV Coordinates]
         |                        |
         └──────────┬─────────────┘
                    ▼
           [Texture Unit (TMU)]
                    |
           [Sampling & Filtering]
                    |
                    ▼
           [Pixel Color Output]

OpenGL Flow:
   glEnable(GL_TEXTURE_2D)
   → glGenTextures()
   → glBindTexture()
   → glTexParameter*()
   → glTexImage2D()
   → glTexCoord*() + glVertex*()
```

Quick check: Want me to:

1. Show complete code for loading an image file as texture?
2. Explain mipmapping and texture filtering in detail?
3. Work through the DDA and Bresenham algorithms with numeric examples?